TASK 2 – HANDWRITTEN DIGIT CLASSIFICATION USING CNN

1. Project Title

Handwritten Digit Classification Using Convolutional Neural Network (CNN)

2. Objective

The objective of this project is to develop a deep learning model that can recognize handwritten digits from images.

A Convolutional Neural Network (CNN) is trained using the MNIST dataset to classify images into one of ten classes, from digit 0 to digit 9.

The trained model is saved and reused to predict digits from new user-provided images.

3. Problem Statement

Handwritten digit recognition is an image classification problem where a computer needs to identify which digit is represented in an input image.

The system should:

- Process handwritten digit images.
- Learn visual patterns from training data.
- Classify digits from 0 to 9.
- Evaluate the model using standard classification metrics.
- Save the trained model for future predictions.
- Predict digits from new input images without retraining.

4. Dataset

The project uses the **MNIST handwritten digit dataset**.

The dataset contains:

- 60,000 training images
- 10,000 testing images
- 10 different classes
- Image size: 28 × 28 pixels
- Grayscale images

The dataset is stored locally in:

```
dataset/mnist.npz
```

The dataset contains the training and testing images along with their corresponding digit labels.

5. Technologies Used

- Python
- TensorFlow
- Keras
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn
- Pillow
- Jupyter Notebook

6. Project Structure

```
task 2
│
├── dataset
│   └── mnist.npz
│
├── sample_images
│   ├── seven.png
│   └── six.png
│
├── mnist_digit_classifier.py
├── predict_digit.py
├── mnist_digit_classifier.ipynb
├── mnist_digit_model.keras
├── requirements.txt
├── README.md
│
├── sample_digits.png
├── accuracy_curve.png
├── loss_curve.png
├── confusion_matrix.png
└── sample_predictions.png
```

7. Data Preprocessing

The MNIST images originally contain pixel values between 0 and 255.

The following preprocessing steps were performed:

1. Images were loaded from the MNIST dataset.

2. Pixel values were normalized from 0–255 to 0–1.
3. A channel dimension was added to make the images suitable for CNN input.
4. The final image shape became:

```
28 × 28 × 1
```

8. CNN Model Architecture

The CNN consists of the following layers:

Layer	Configuration
Input	28 × 28 × 1
Conv2D	32 filters, 3 × 3, ReLU
MaxPooling2D	2 × 2
Conv2D	64 filters, 3 × 3, ReLU
MaxPooling2D	2 × 2
Flatten	Converts feature maps into a vector
Dense	128 neurons, ReLU
Dropout	0.5
Output	10 neurons, Softmax

The model contains **225,034 trainable parameters**.

9. Model Training

The model was trained for **5 epochs** using:

- Optimizer: Adam
- Loss function: Sparse Categorical Crossentropy
- Batch size: 64
- Validation split: 10%

Training performance improved across the five epochs.

Epoch	Training Accuracy	Validation Accuracy
1	92.34%	98.42%
2	97.31%	98.72%
3	98.01%	98.97%
4	98.44%	99.10%
5	98.66%	99.15%

10. Model Evaluation

The trained model was evaluated on the 10,000-image MNIST test dataset.

Test Loss: 0.0256

Test Accuracy: 99.16%

The model achieved approximately 99% precision, recall, and F1-score across the ten digit classes.

11. Classification Results

Digit	Precision	Recall	F1-score
0	0.99	0.99	0.99
1	0.99	1.00	1.00
2	1.00	0.99	0.99
3	0.99	1.00	0.99
4	1.00	0.99	1.00
5	0.99	0.99	0.99
6	1.00	0.99	0.99
7	0.98	1.00	0.99
8	0.99	0.99	0.99
9	0.99	0.99	0.99

Overall accuracy on the test set was **99.16%**.

12. Training and Evaluation Visualizations

The project generates the following visualizations:

Sample Digits

`sample_digits.png` displays examples of handwritten digits from the MNIST dataset.

Accuracy Curve

`accuracy_curve.png` shows training and validation accuracy across the five epochs.

Loss Curve

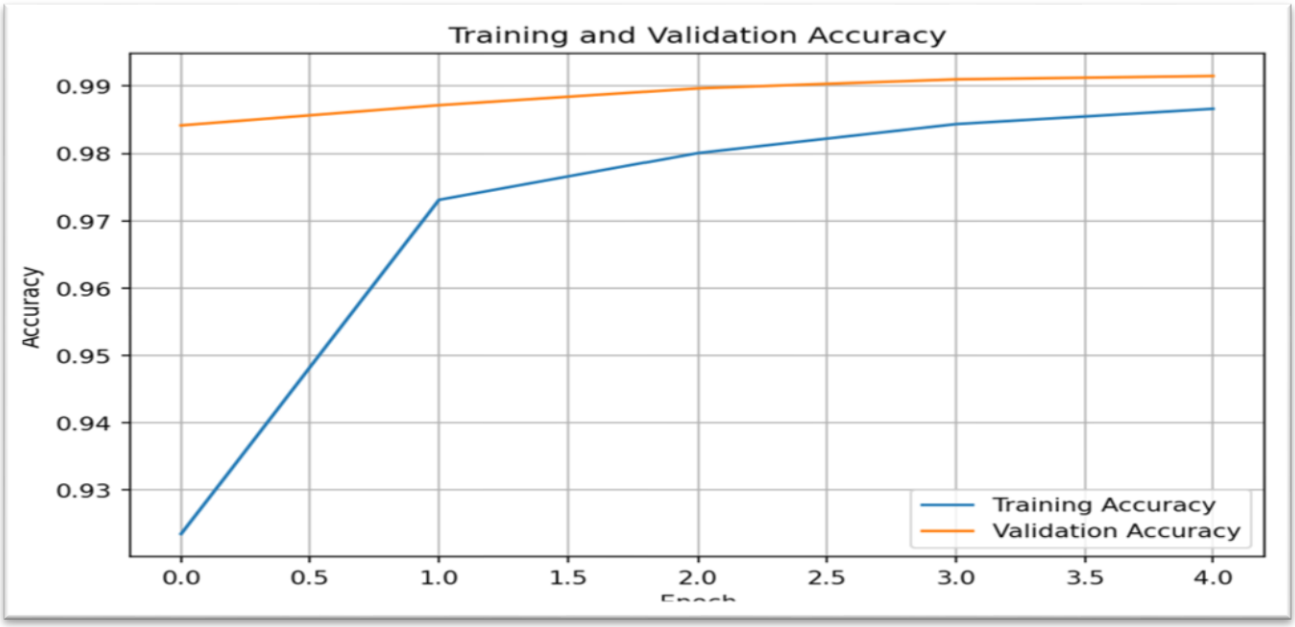
`loss_curve.png` shows training and validation loss during training.

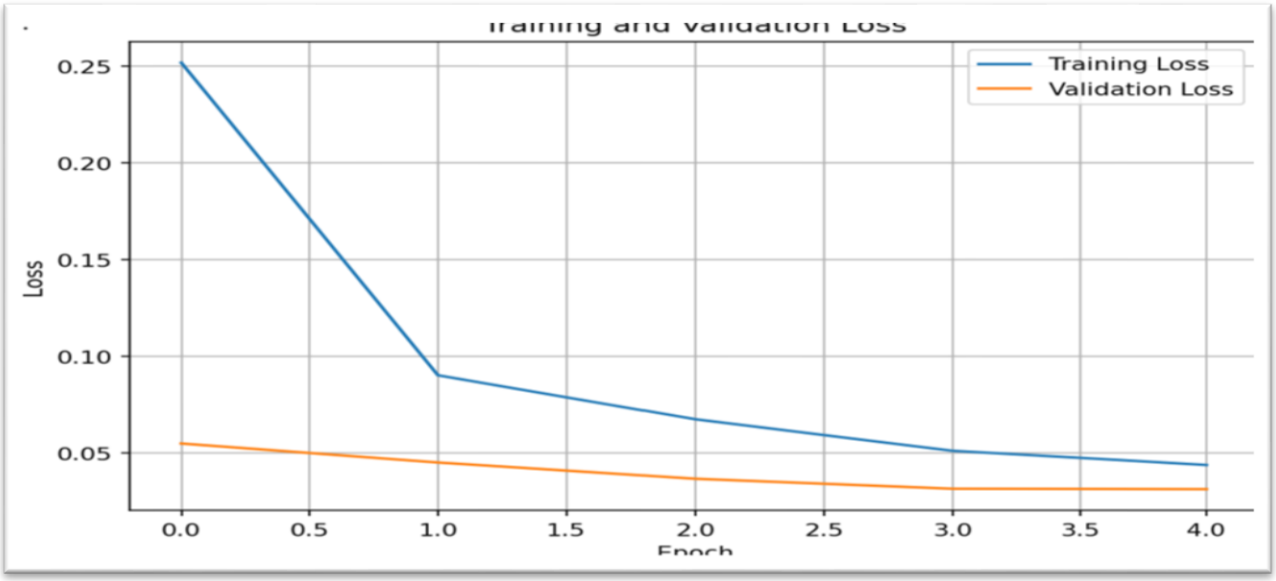
Confusion Matrix

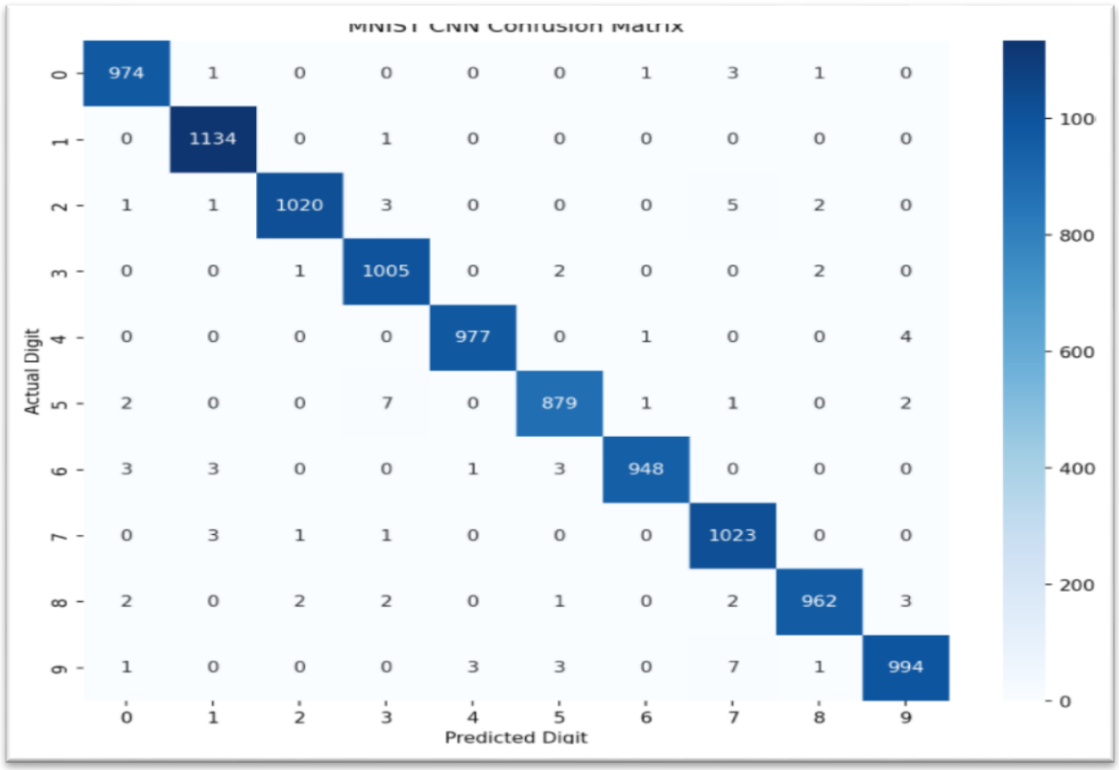
`confusion_matrix.png` shows the classification performance for each digit class.

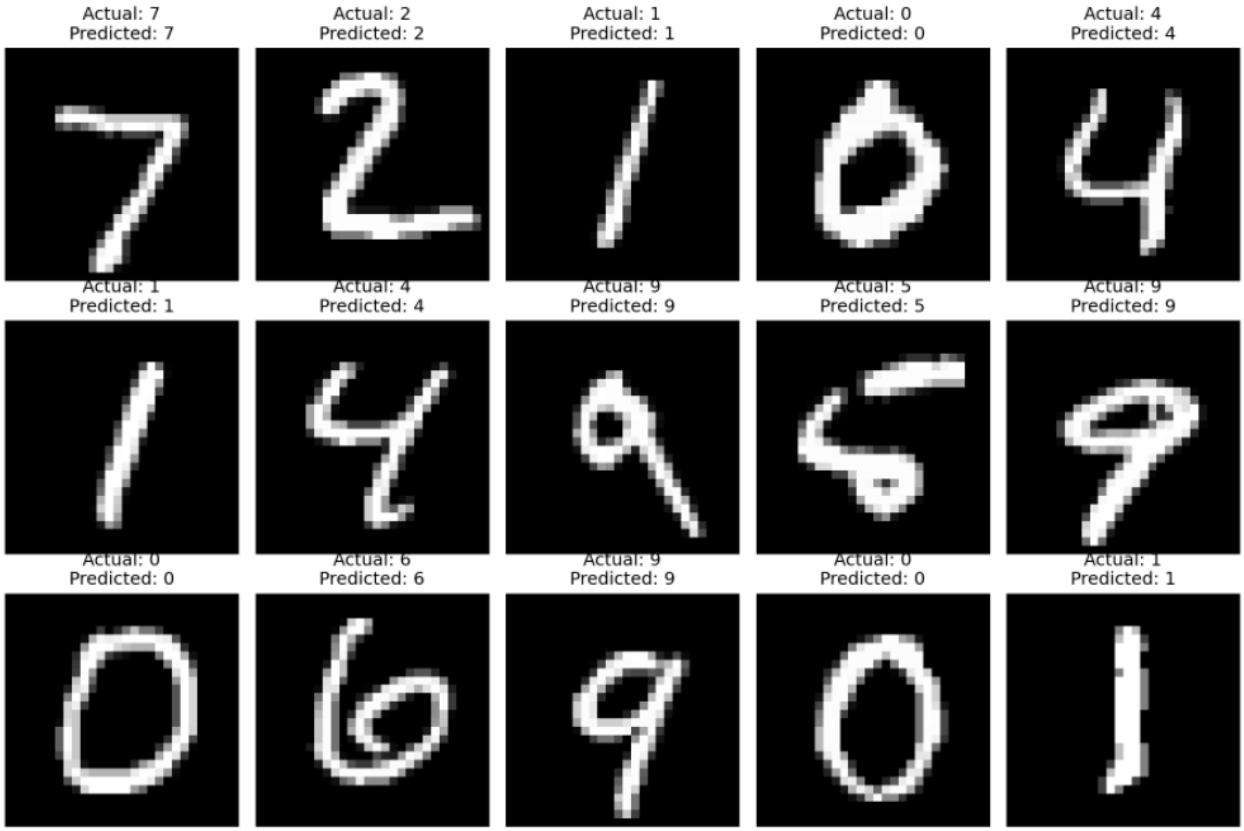
Sample Predictions

`sample_predictions.png` displays example test images along with their actual and predicted labels.






MNIST CNN Confusion Matrix
Actual Digit
Predicted Digit
0 974 1 0 0 0 0 1 3 1 0
1 0 1134 0 1 0 0 0 0 0 0
2 1 1 1020 3 0 0 0 5 2 0
3 0 0 1 1005 0 2 0 0 2 0
4 0 0 0 0 977 0 1 0 0 4
5 2 0 0 7 0 879 1 1 0 2
6 3 3 0 0 1 3 948 0 0 0
7 0 3 1 1 0 0 0 1023 0 0
8 2 0 2 2 0 1 0 2 962 3
9 1 0 0 0 3 3 0 7 1 994
0 1 2 3 4 5 6 7 8 9
100
800
600
400
200
0


Actual: 7
Predicted: 7
Actual: 2
Predicted: 2
Actual: 1
Predicted: 1
Actual: 0
Predicted: 0
Actual: 4
Predicted: 4
Actual: 1
Predicted: 1
Actual: 4
Predicted: 4
Actual: 9
Predicted: 9
Actual: 5
Predicted: 5
Actual: 9
Predicted: 9
Actual: 0
Predicted: 0
Actual: 6
Predicted: 6
Actual: 9
Predicted: 9
Actual: 0
Predicted: 0
Actual: 1
Predicted: 1

13. Saved Model

After training, the CNN model was saved as:

```
mnist_digit_model.keras
```

The saved model can be loaded without retraining.

The separate `predict_digit.py` program loads this saved model and performs inference on a new image.

14. New Image Prediction

A separate prediction program was created to test the trained model with user-provided images.

The program is executed using:

```
python predict_digit.py
```

The user enters the image path when prompted.

Example:

```
Enter the image path: sample_images/seven.png
```

The program preprocesses the image and passes it to the saved CNN model.

The project was tested using sample images containing handwritten-style digits **6 and 7**.

This demonstrates that the saved model can be reused for inference without training again.

15. Reproducibility

Install the required dependencies using:

```
pip install -r requirements.txt
```

Train the model using:

```
python mnist_digit_classifier.py
```

Run prediction on a new image using:

```
python predict_digit.py
```

The complete project source code and instructions are provided in the project repository.

16. Deliverables

The completed project contains:

- `mnist_digit_classifier.py`
- `mnist_digit_classifier.ipynb`
- `predict_digit.py`
- `mnist_digit_model.keras`
- `dataset/mnist.npz`
- `requirements.txt`
- `README.md`
- Training accuracy plot
- Training loss plot
- Confusion matrix
- Sample predictions
- Sample digit visualization

17. Conclusion

A CNN-based handwritten digit classification system was successfully developed using the MNIST dataset.

The model achieved a **99.16% test accuracy** after five training epochs. The classification results show approximately 99% precision, recall, and F1-score across the ten digit classes.

The trained model was saved as a Keras model and successfully reused for prediction on new input images. This demonstrates the complete workflow of dataset preparation, CNN training, evaluation, model saving, and image inference.